

Fine-Tuning of Model

```
In [1]: import pandas as pd
```

```
In [2]: from sklearn.datasets import load_iris
```

```
In [5]: iris=load_iris()
```

```
In [6]: X = iris.data           # shape (150, 4) → 4 features  
        y = iris.target
```

```
In [7]: import pandas as pd
```

```
df = pd.DataFrame(X, columns=iris.feature_names)  
df['target'] = y  
  
print(df.head())
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
0	5.1	3.5	1.4	0.	0
1	4.9	3.0	1.4	0.	0
2	4.7	3.2	1.3	0.	0
3	4.6	3.1	1.5	0.	0
4	5.0	3.6	1.4	0.	0

```
In [8]: df
```

```
Out[8]:
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	2
146	6.3	2.5	5.0	1.9	2
147	6.5	3.0	5.2	2.0	2
148	6.2	3.4	5.4	2.3	2
149	5.9	3.0	5.1	1.8	2

150 rows × 5 columns

```
In [10]: from sklearn.model_selection import train_test_split
```

```
In [13]: X_train,X_test,y_tain,y_test=train_test_split(df.drop('target',axis=1),df
```

```
In [19]: y_tain.shape
```

```
Out[19]: (120,)
```

```
In [20]: y_test.shape
```

```
Out[20]: (30,)
```

```
In [22]: df['target'].value_counts()
```

```
Out[22]: target
0      50
1      50
2      50
Name: count, dtype: int64
```

```
In [35]: from sklearn.model_selection import GridSearchCV
from sklearn.svm import SVC
```

```
In [54]: param_grid = {
    'C': [0.5, 1.0,10, 15, 20], # Note: C=0 is generally invalid for SVC,
    'kernel': ['rbf', 'linear']
}

cff = GridSearchCV(SVC(gamma='auto'), param_grid, cv=5, return_train_scor
```

```
In [55]: cff.fit(iris.data,iris.target)
```

Out[55]:

```
  ▶ GridSearchCV ⓘ ?  
  ▶ best_estimator_: SVC  
    ▶ SVC ?
```

In [56]: `cff.cv_results_`

```

Out[56]: {'mean_fit_time': array([0.00443077, 0.00408597, 0.00378718, 0.00336127,
0.00337195,
      0.00267048, 0.00265498, 0.00185852, 0.00134163, 0.00099015]),
'std_fit_time': array([1.31171188e-03, 9.02372406e-04, 1.46260236e-04,
8.32684388e-05,
      2.51325527e-04, 8.18885601e-05, 2.11456716e-05, 3.63157608e-04,
1.49947917e-04, 7.31821547e-05]),
'mean_score_time': array([0.00259042, 0.00283566, 0.00265074, 0.0021922
1, 0.00203357,
      0.0017189 , 0.00166879, 0.00118046, 0.00082932, 0.00062928]),
'std_score_time': array([5.27346133e-04, 7.66240913e-04, 5.32983718e-0
4, 1.84211483e-04,
      8.10988568e-05, 2.07139510e-05, 6.92087700e-06, 2.33385432e-04,
8.19262024e-05, 3.47219971e-05]),
'param_C': masked_array(data=[0.5, 0.5, 1.0, 1.0, 10.0, 10.0, 15.0, 15.
0, 20.0, 20.0],
      mask=[False, False, False, False, False, False, False, Fal
se,
      False, False],
      fill_value=1e+20),
'param_kernel': masked_array(data=['rbf', 'linear', 'rbf', 'linear', 'r
bf', 'linear',
      'rbf', 'linear', 'rbf', 'linear'],
      mask=[False, False, False, False, False, False, False, Fal
se,
      False, False],
      fill_value=np.str_('?'),
      dtype=object),
'params': [{ 'C': 0.5, 'kernel': 'rbf'},
{ 'C': 0.5, 'kernel': 'linear'},
{ 'C': 1.0, 'kernel': 'rbf'},
{ 'C': 1.0, 'kernel': 'linear'},
{ 'C': 10, 'kernel': 'rbf'},
{ 'C': 10, 'kernel': 'linear'},
{ 'C': 15, 'kernel': 'rbf'},
{ 'C': 15, 'kernel': 'linear'},
{ 'C': 20, 'kernel': 'rbf'},
{ 'C': 20, 'kernel': 'linear'}],
'split0_test_score': array([0.96666667, 0.96666667, 0.96666667, 0.96666
667, 0.96666667,
      1.          , 0.96666667, 1.          , 0.96666667, 1.          ]),
'split1_test_score': array([1., 1., 1., 1., 1., 1., 1., 1., 1., 1.]),
'split2_test_score': array([0.96666667, 1.          , 0.96666667, 0.96666
667, 0.96666667,
      0.9          , 0.93333333, 0.9          , 0.9          , 0.9          ]),
'split3_test_score': array([0.96666667, 0.96666667, 0.96666667, 0.96666
667, 0.96666667,
      0.96666667, 0.96666667, 0.93333333, 0.96666667, 0.93333333]),
'split4_test_score': array([1., 1., 1., 1., 1., 1., 1., 1., 1., 1.]),
'mean_test_score': array([0.98          , 0.98666667, 0.98          , 0.98
, 0.98
,
      0.97333333, 0.97333333, 0.96666667, 0.96666667, 0.96666667]),
'std_test_score': array([0.01632993, 0.01632993, 0.01632993, 0.0163299
3, 0.01632993,
      0.03887301, 0.02494438, 0.0421637 , 0.03651484, 0.0421637 ]),
'rank_test_score': array([2, 1, 2, 2, 2, 6, 6, 9, 8, 9], dtype=int32)}

```

```

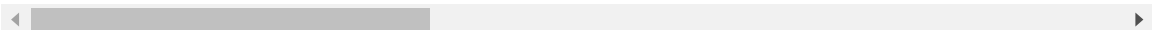
In [57]: results=pd.DataFrame(cff.cv_results_)

```

```
In [58]: results
```

Out[58]:

	mean_fit_time	std_fit_time	mean_score_time	std_score_time	param_C	param_kern
0	0.004431	0.001312	0.002590	0.000527	0.5	l
1	0.004086	0.000902	0.002836	0.000766	0.5	line
2	0.003787	0.000146	0.002651	0.000533	1.0	l
3	0.003361	0.000083	0.002192	0.000184	1.0	line
4	0.003372	0.000251	0.002034	0.000081	10.0	l
5	0.002670	0.000082	0.001719	0.000021	10.0	line
6	0.002655	0.000021	0.001669	0.000007	15.0	l
7	0.001859	0.000363	0.001180	0.000233	15.0	line
8	0.001342	0.000150	0.000829	0.000082	20.0	l
9	0.000990	0.000073	0.000629	0.000035	20.0	line



```
In [59]: results[['param_C', 'param_kernel', 'mean_test_score']]
```

```
Out[59]:
```

	param_C	param_kernel	mean_test_score
0	0.5	rbf	0.980000
1	0.5	linear	0.986667
2	1.0	rbf	0.980000
3	1.0	linear	0.980000
4	10.0	rbf	0.980000
5	10.0	linear	0.973333
6	15.0	rbf	0.973333
7	15.0	linear	0.966667
8	20.0	rbf	0.966667
9	20.0	linear	0.966667

```
In [47]: parameters=dir(cff)
```

Scikit-learn Object Reference (Quick Guide)

1 Common Special / Dunder Methods

- `__init__` → Constructor for the object
- `__repr__`, `__str__` → Object representation (print/display)
- `__class__` → Class of the object
- `__dict__` → Attribute dictionary of the object
- `__doc__` → Documentation string
- `__eq__`, `__ne__`, `__lt__`, `__le__`, `__gt__`, `__ge__` → Comparison operators
- `__getattr__`, `__setattr__`, `__delattr__` → Attribute access

2 Sklearn-Specific Internal Methods

- `_validate_params` → Check parameter validity
- `_get_param_names` → Return parameter names of estimator
- `_run_search` → Run the hyperparameter search
- `_format_results` → Format results for `cv_results_`
- `_sklearn_tags` → Metadata for estimator properties
- `_sklearn_clone` → Used for cloning the estimator
- `_check_refit_for_multimetric` → Internal check during multi-metric scoring

3 Hyperparameter Search / GridSearchCV Attributes

- `best_estimator_` → Best model found after search
 - `best_params_` → Hyperparameters of best model
 - `best_score_` → Best cross-validation score
 - `best_index_` → Index of best parameters in `cv_results_`
 - `cv_results_` → Full dictionary of cross-validation results
 - `cv` → Cross-validation splitting strategy
 - `param_grid` → Grid of parameters to search
 - `scoring` → Scoring metric used
 - `refit` → Refit best estimator on full training data
 - `refit_time_` → Time to refit the best estimator
-

4 Estimator / Model Methods

- `fit(X, y)` → Train the model
 - `predict(X)` → Predict using the trained model
 - `score(X, y)` → Compute accuracy or R^2 score
 - `decision_function(X)` → Decision function output (for classifiers)
 - `get_params()` → Get current parameters
 - `set_params(**params)` → Set new hyperparameters
 - `classes_` → List of class labels (for classification)
 - `n_features_in_` → Number of features seen during training
-

5 Performance / Multi-Metric Attributes

- `multimetric_` → Indicates if multiple metrics are used
 - `return_train_score` → Return train scores in CV results
 - `scorer_` → Scorer function used
 - `_check_scorers_accept_sample_weight` → Internal scorer check
-

6 Parallel / Runtime Attributes

- `n_jobs` → Number of parallel jobs
- `verbose` → Verbosity level of output
- `pre_dispatch` → Pre-dispatch configuration for parallelism

```
In [50]: cff.best_estimator_
```

```
Out[50]: ▼ SVC ⓘ ?
          ► Parameters
```

```
In [52]: cff.best_score_
```

```
Out[52]: np.float64(0.9866666666666667)
```

```
In [53]: cff.best_index_
```

```
Out[53]: np.int64(1)
```

```
In [ ]:
```